Spring Core_Maven

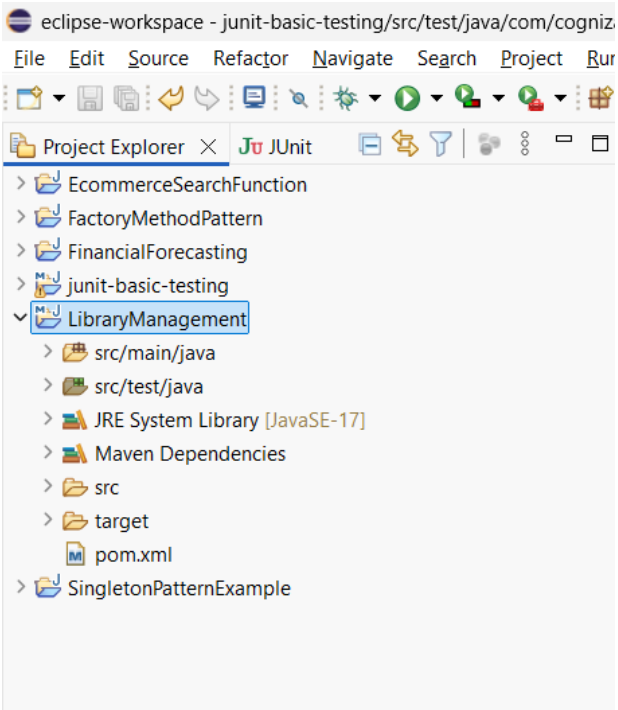
Exercise 1: Configuring a Basic Spring Application

Scenario:
Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.
Steps:

1. **Set Up a Spring Project:**
   - Create a Maven project named **LibraryManagement**.
   - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
   - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
   - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
   - Create a package **com.library.service** and add a class **BookService**.
   - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
   - Create a main class to load the Spring context and test the configuration.

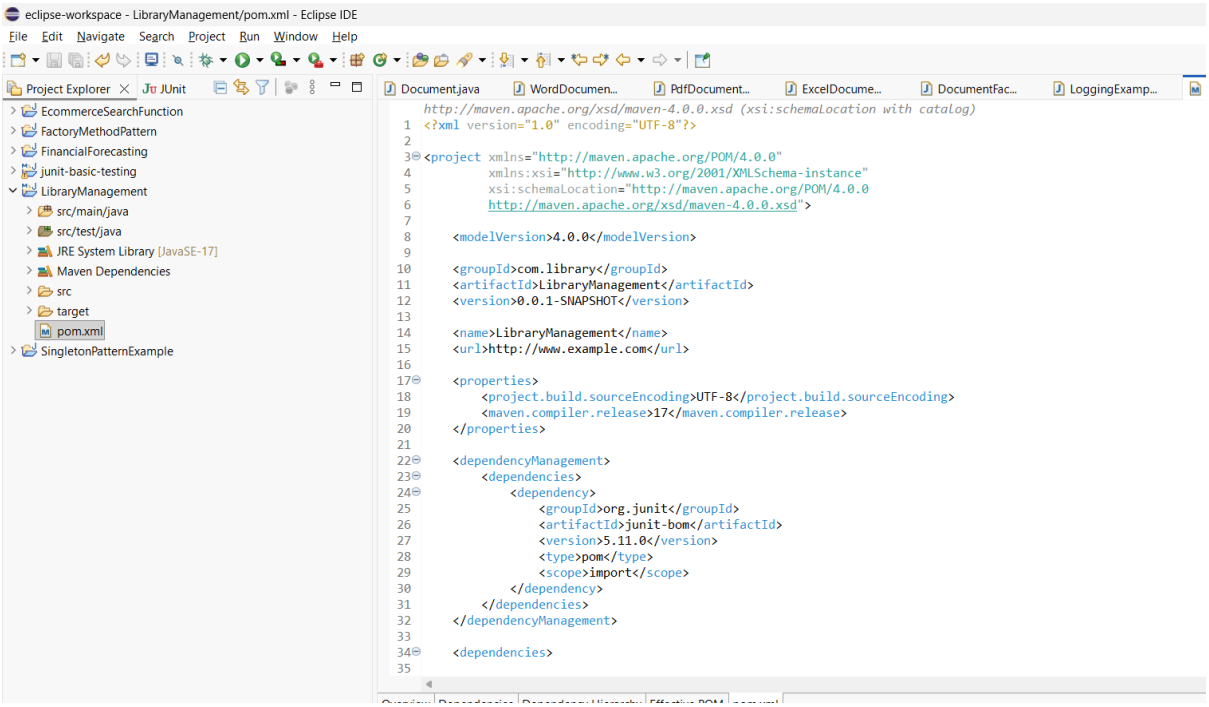
Step 1: Open the Existing Maven Project

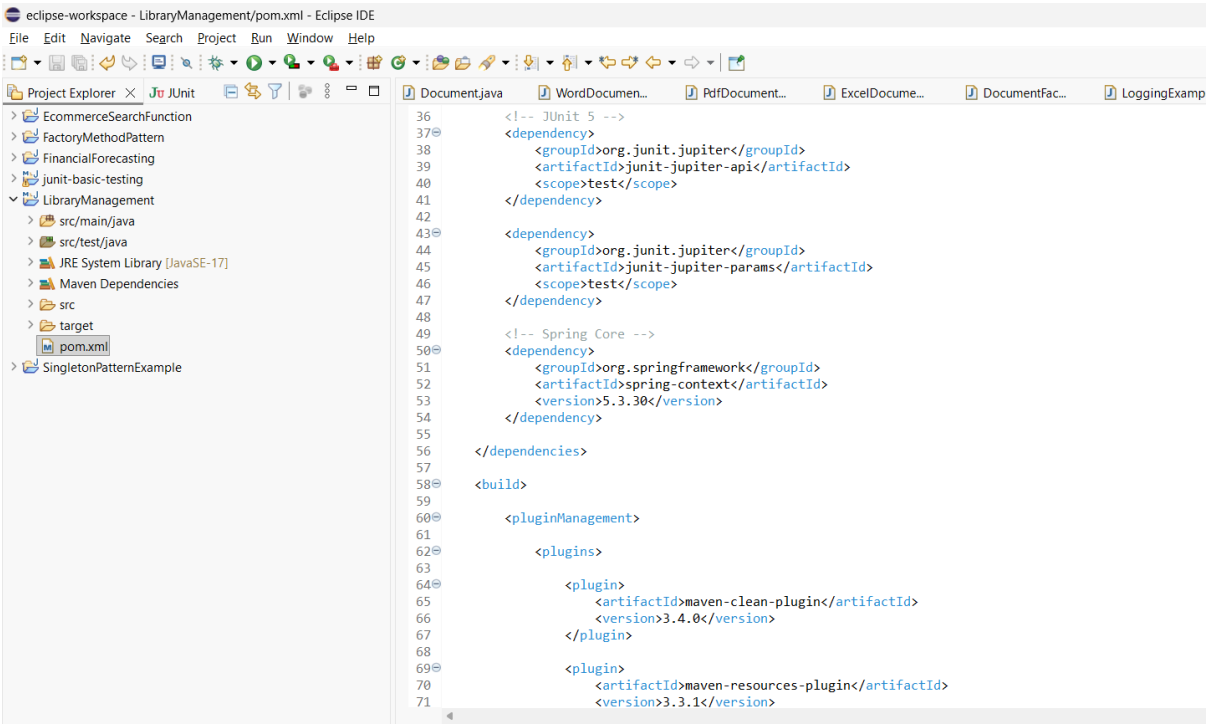
Opened the existing Maven project junit-basic-testing in Eclipse IDE.



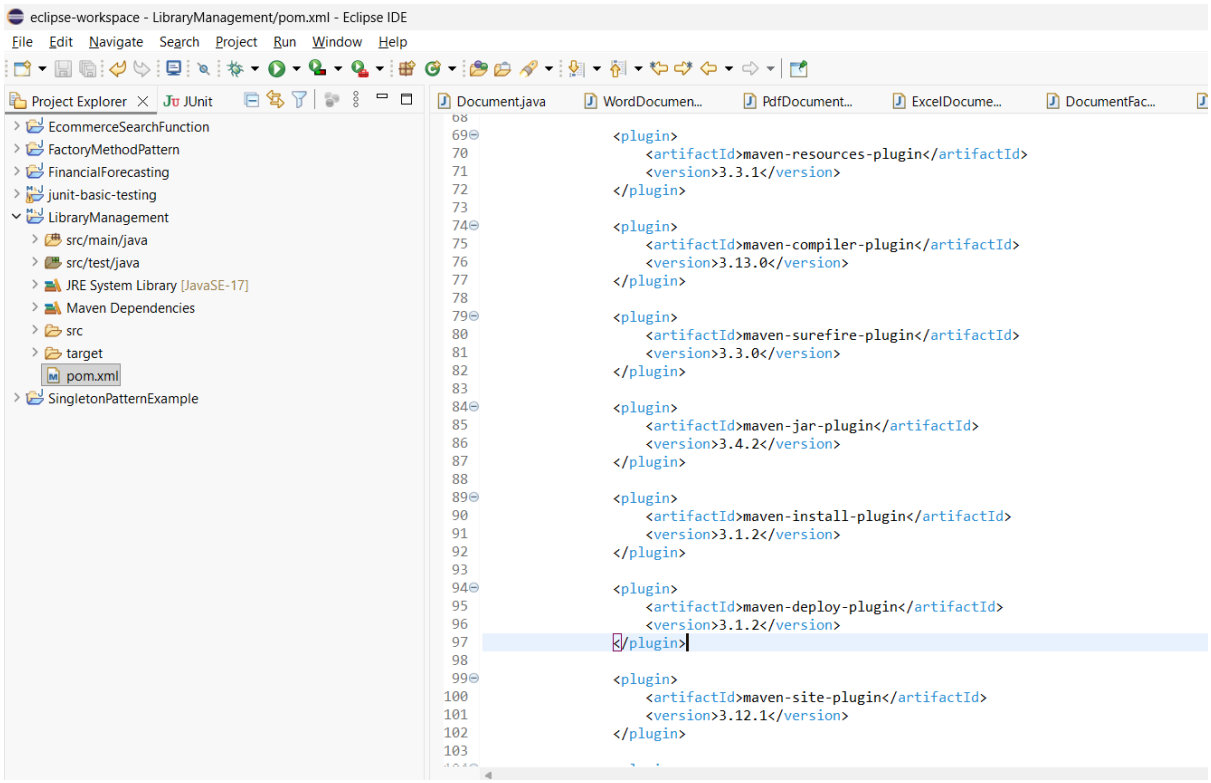
Step 2: Open pom.xml

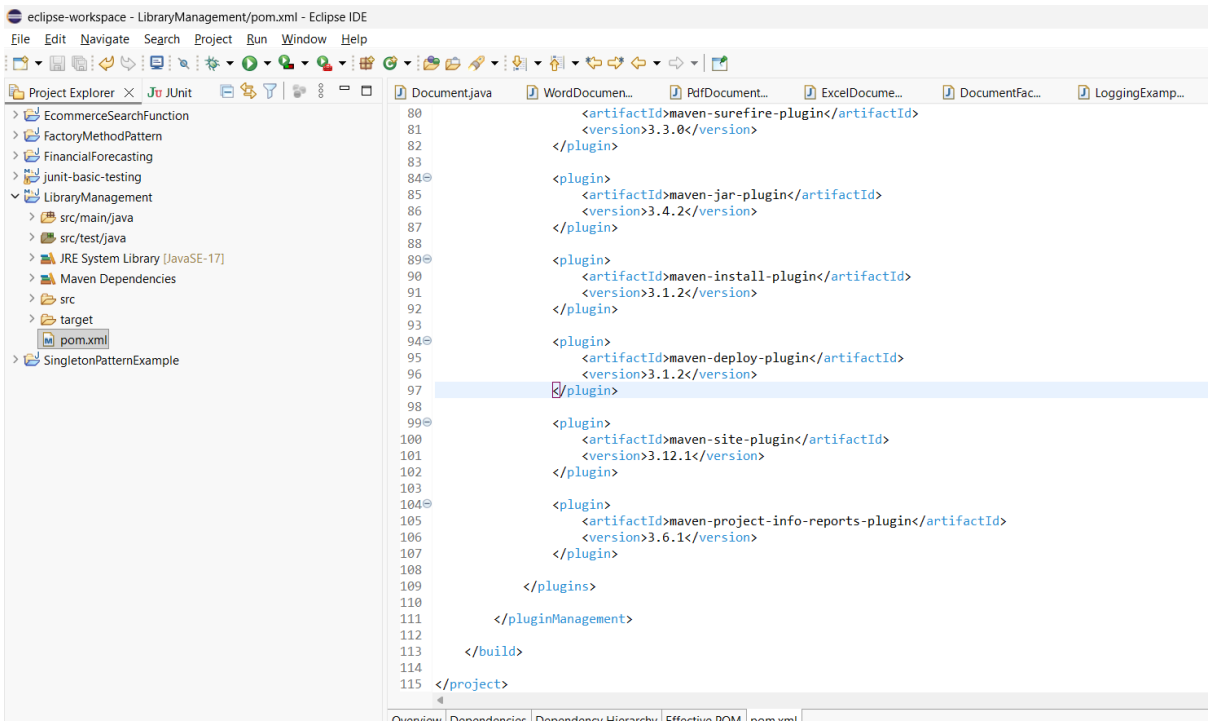
Opened the pom.xml file and added the required Spring Core dependency to enable Spring Framework support in the project.





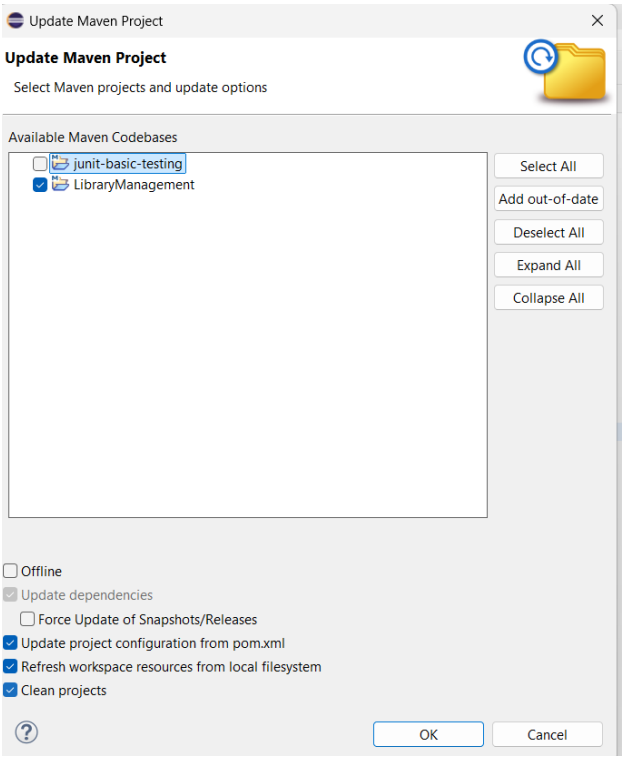
COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE





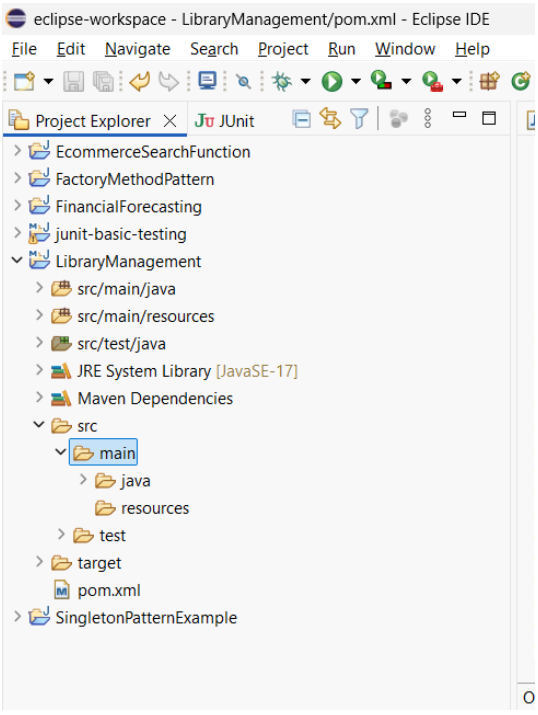
Step 3: Update the Maven Project

Updated the Maven project to download all Spring dependencies and refresh the project configuration.



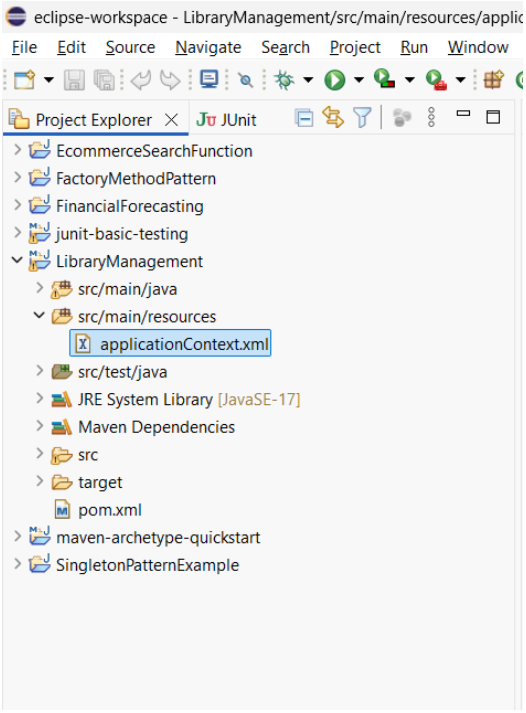
Step 4: Verify the Project Structure

Verified that the Maven project contains the required directory structure, including src/main/java, src/main/resources, and pom.xml.



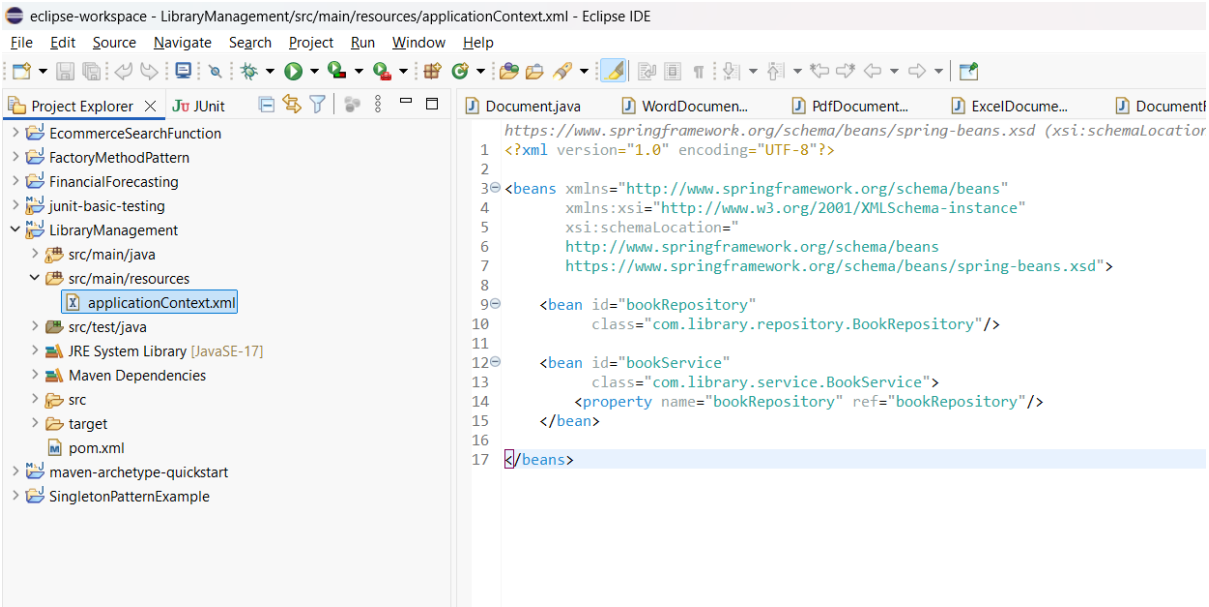
Step 5: Create applicationContext.xml

Created the applicationContext.xml file inside the src/main/resources folder to configure Spring beans.



Step 6: Configure Spring Beans

Configured the **BookRepository** and **BookService** beans in **applicationContext.xml** using XML-based Spring configuration.

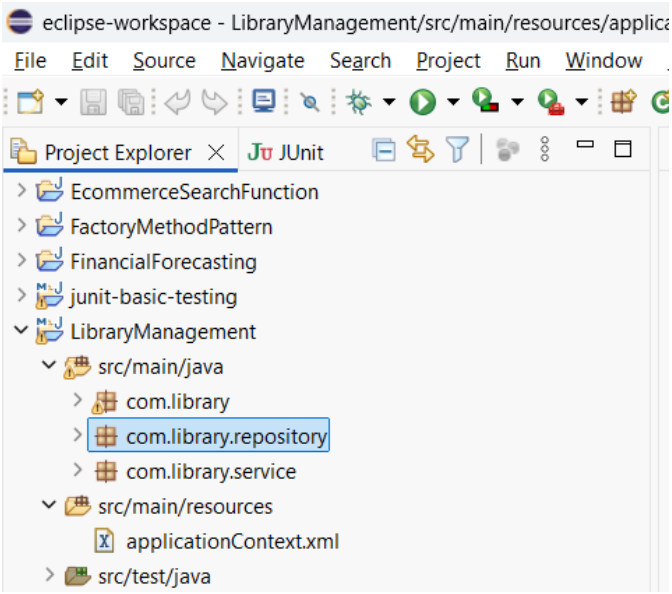


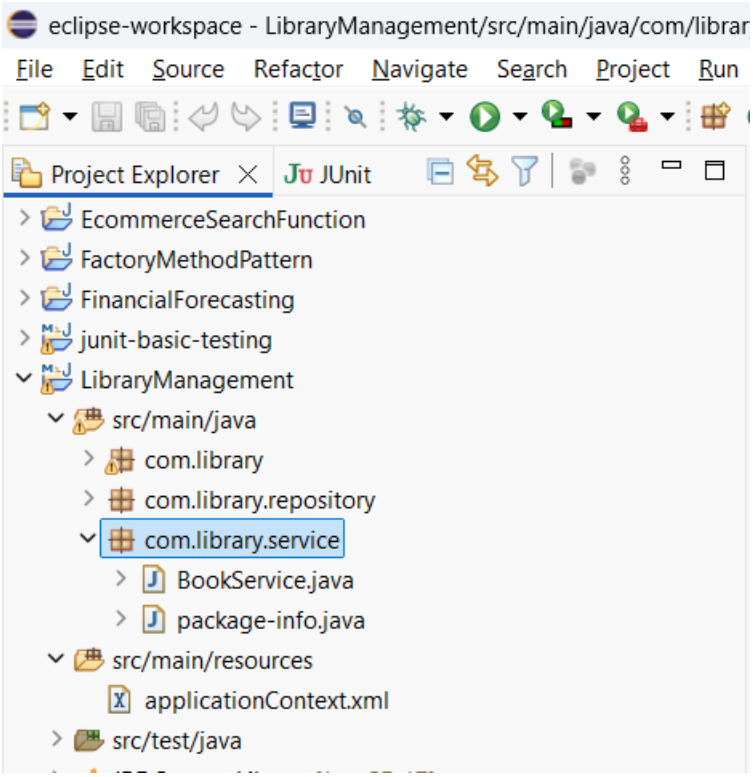
Step 7: Create Java Packages

Created the required packages:

- **com.library.repository**
- **com.library.service**

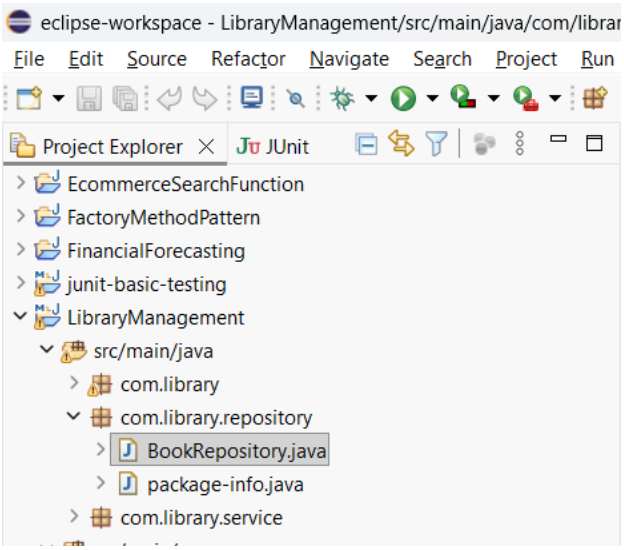
to organize the application classes.

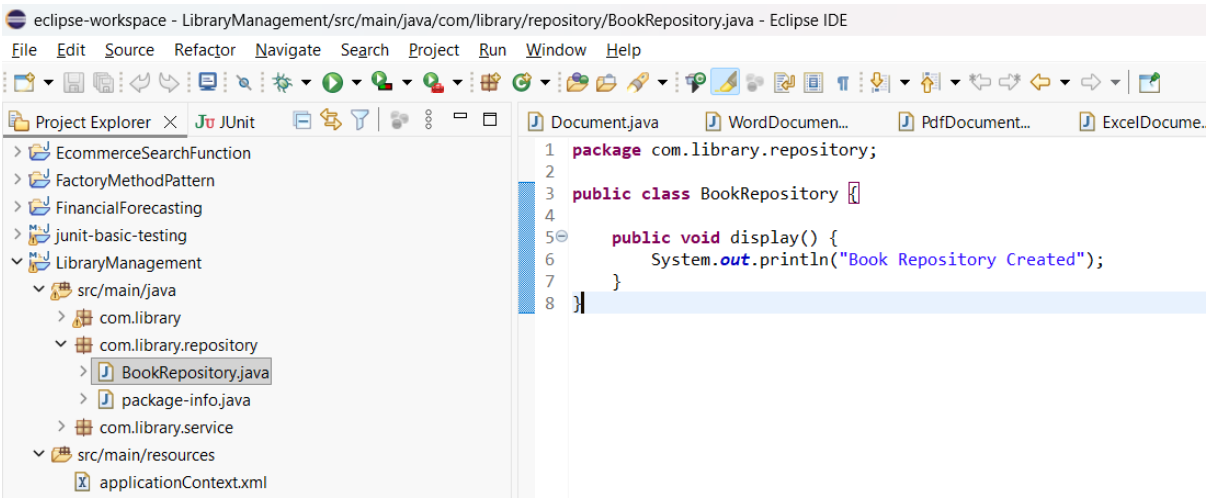




Step 8: Create BookRepository Class

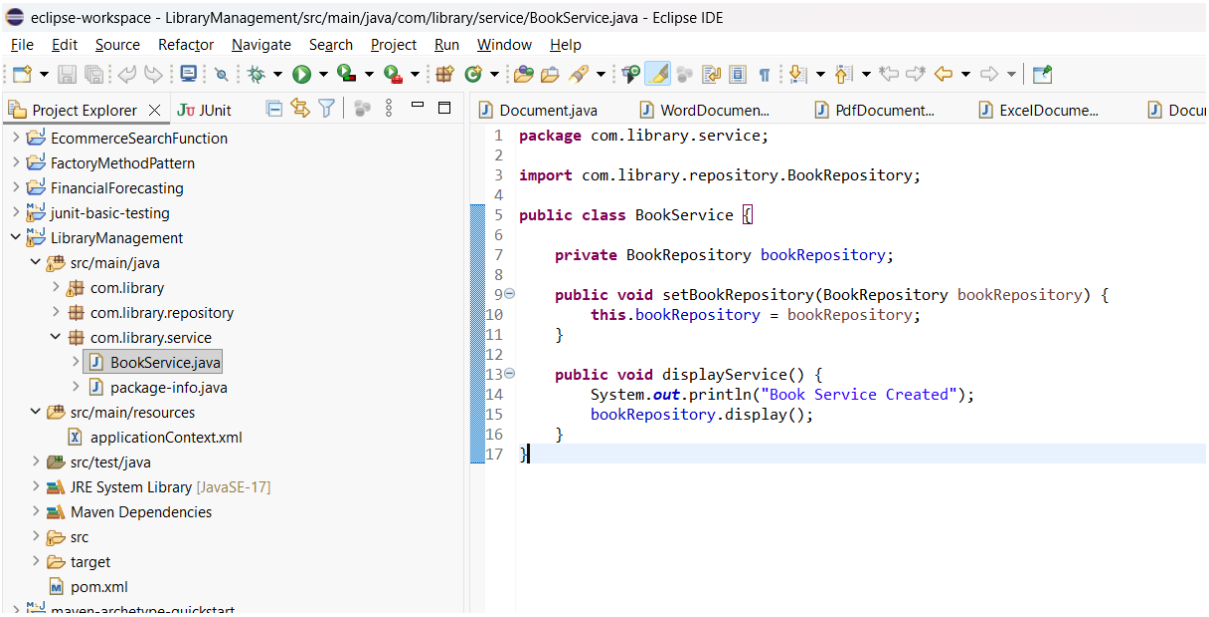
Implemented the **BookRepository** class, which represents the repository layer responsible for data-related operations.





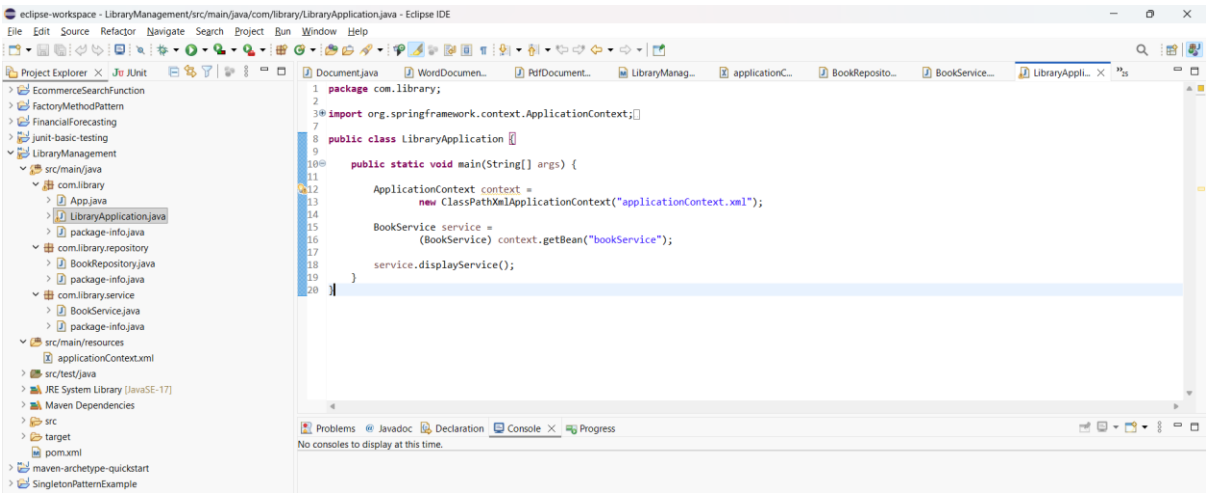
Step 9: Create BookService Class

Implemented the **BookService** class and injected the **BookRepository** bean using setter injection.



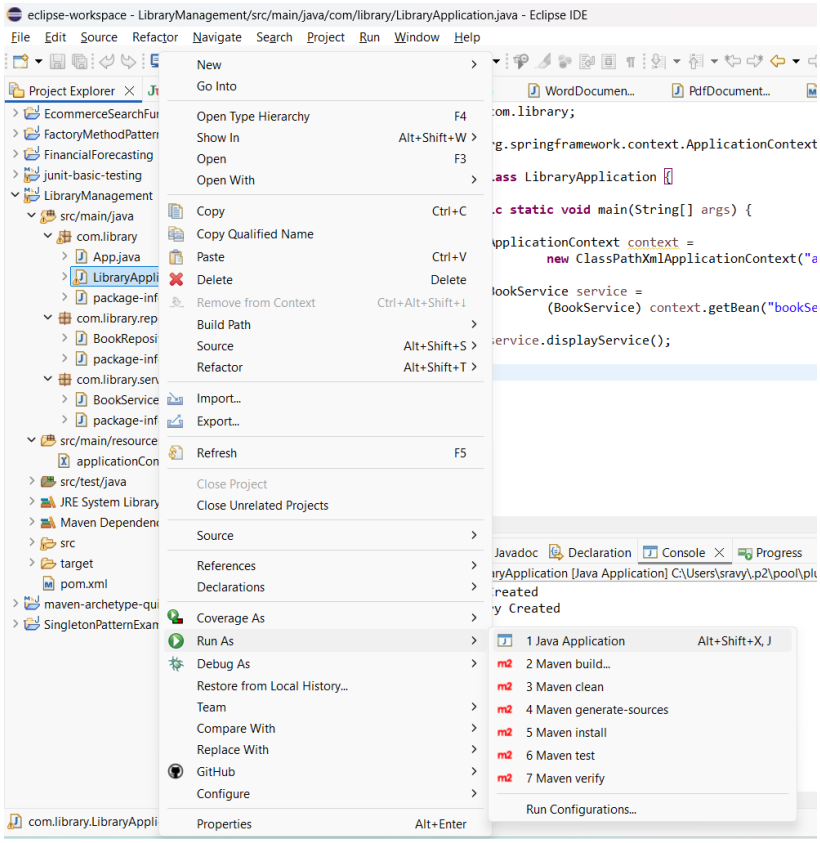
Step 10: Create the Main Class

Created the **LibraryApplication** class to load the Spring Application Context and retrieve the configured bean.



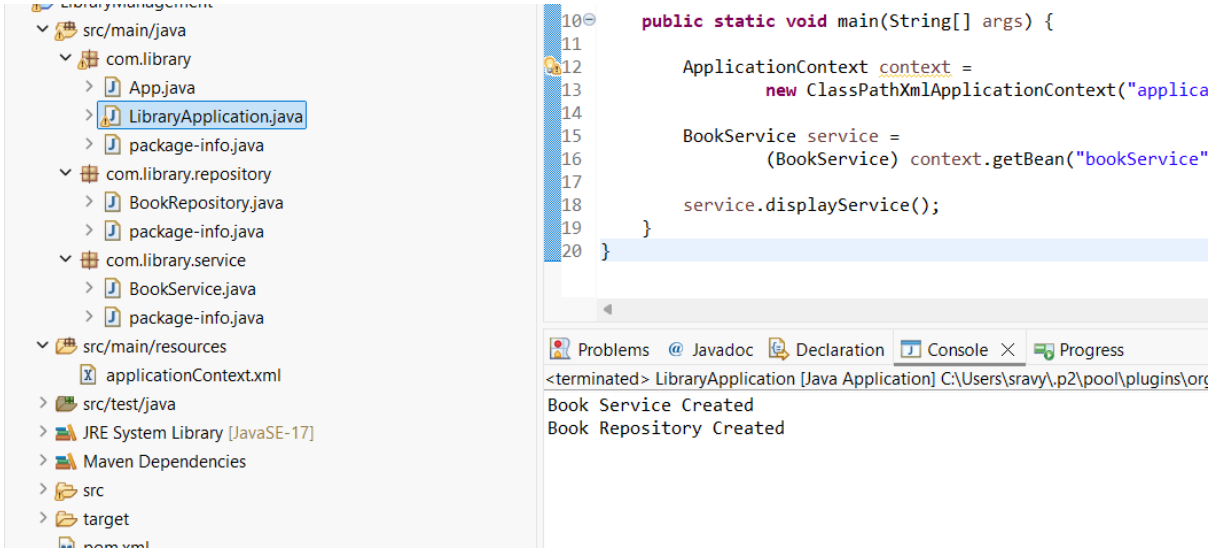
Step 11: Execute the Application

Executed the **LibraryApplication.java** file as a Java Application to verify that the Spring configuration was successful.



Output

The above output confirms that the Spring Application Context successfully created and managed the configured beans.



Result

The Library Management application was successfully configured using the Spring Framework and Maven. The application loaded the XML configuration file, created the **BookRepository** and **BookService** beans, injected the dependencies correctly, and executed successfully.